C-Cord Project

Redes de Computadores — 2025/2026
Universidade de Coimbra — DEEC

Team Manager: Gonçalo Peres
Account Manager: João Cascão
Software Manager: Gonçalo Peres
Risk and Testing Manager: Angeles de Jauregui
Quality Manager: Flavia Salta
Development team: Dinis Madeira
Gabriel da Cruz

Team Manager Report

Project Planning

In order to optimize the productivity of the 7 group members and meet the requirements of the Computer Networks evaluation, the project was structured into three development phases, with team restructuring occurring at each stage.

Phase 1: Blocking Sockets and Planning (Until 14/04/2026)

In this initial phase, the 7-member group works together to set up the base architecture of the Blocking Sockets and prepare the project documentation.

- **Member 1: Team Manager** - Overall team coordination, creation of the Gantt Chart, general and individual planning for the 3 stages, and responsible for the oral presentation in the first evaluation class.
- **Member 2: Software Manager** - Configuration of GitHub branches and Trello.
- **Member 3: Account Manager** - Gathering functional and non-functional requirements (F1, F2, and F3), designing operating diagrams, and creating visual application mockups.
- **Member 4: Risk and Testing Manager** - Preparation of the risk plan for the blocking architecture and creation of the mitigation and testing plan.
- **Member 5: Quality Manager** - Writing the state-of-the-art analysis and responsible for compiling and reviewing the overall report for this stage.
- **Members 6 and 7: Development Team** - C programming focused on implementing the blocking Socket API and features F1 (Register), F2 (Simple Authentication), and F3 (Echo & Info Retrieval).

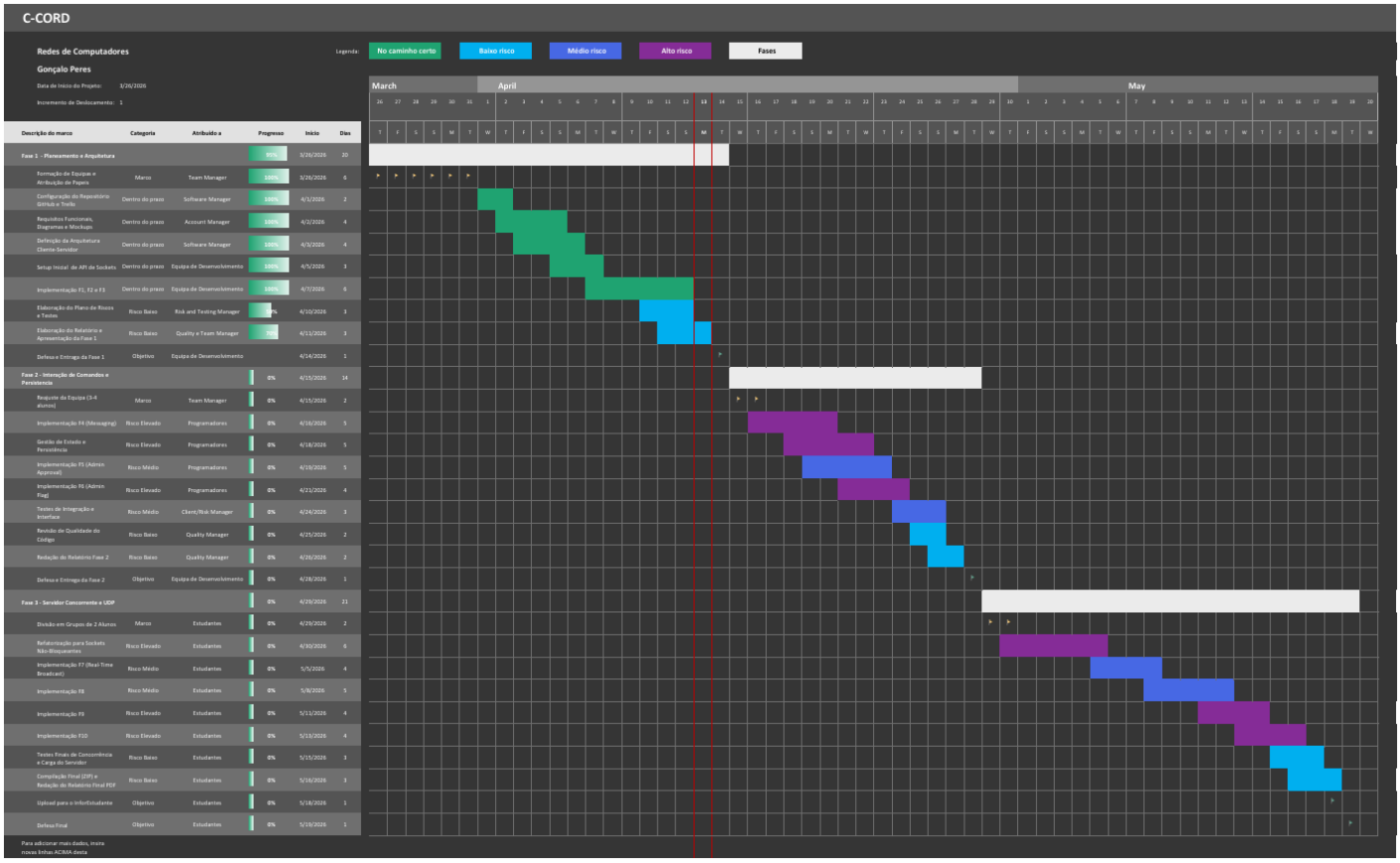
Phase 2: Command Interaction and Data Persistence (From 15/04/2026 to 28/04/2026)

Following the guidelines, the 7-member team will be divided into **two independent sub-teams** (one with 4 students and another with 3 students). The planning below reflects the organization of one of these teams (with roles mirrored in the second team).

- **Member A: Team & Software Manager** - Assumes technical coordination, readjusts the Gantt Chart, manages GitHub merges, and will be responsible for the Phase 2 oral presentation.
- **Member B: Client, Quality, Risk & Testing Manager** - Acts as the customer interface, manages code testing for user approvals, and ensures the quality of the interim report.
- **Members C and D: Team of Programmers** - Focused purely on implementing C code for features F4 (User Directory and Messaging), F5 (Admin Management/Approval), and F6 (Admin Role / DELETEUSER command), as well as message state persistence.

Phase 3: Real-Time Concurrency and UDP (From 29/04/2026 to 18/05/2026)
In this critical phase, evaluation and development will be carried out in **pairs** (groups of 2 students). Given the initial 7 people, three pairs will be formed, alongside one individual working group (or adapted according to the professor's instructions).

- **Members 1 and 2:** Work in *Pair Programming*, sharing management, reporting, and coding responsibilities equally.
- **Pair Tasks:** Total refactoring of the server to abandon Blocking Sockets using the select() or poll function. Shared implementation of Real-Time Broadcast (F7), multi-channel support (F8), and P2P file exchange via UDP sockets (F9). Both will be responsible for the pair's final report, to be delivered on May 18th.



Account Manager Report

This section defines and justifies the functional and non-functional requirements of the system, as well as its operating diagrams and application mockups, for Phase 1 of the C Cord project.

1. Functional Requirements

F1 – User Registration: The system must allow users to register with a unique username and password, stored persistently in a file.

Justification: This ensures persistent identity and prevents duplicate accounts.

F2 – Authentication: The system must validate login credentials against stored user data and reject invalid access.

Justification: This guarantees that only authorized users can access the system.

F3 – Echo & Info Retrieval: The system must support ECHO and GET_INFO commands, returning the same message or server status.

Justification: This validates correct client-server communication and system responsiveness.

2. Non-Functional Requirements

Performance: The system should respond to client requests within 1 second.

Reliability: User data must be immediately written to disk to prevent data loss.

Security: Passwords are stored in plain text (acknowledged limitation for Phase 1).

Usability: The client interface should provide a clear and simple menu-based interaction.

Maintainability: The system should be modular, separating client and server logic.

Scalability: The system currently supports only one client at a time due to blocking sockets.

3. Requirements

F1 → User Registration requirements
F2 → Authentication requirements
F3 → Echo & Info requirements

4. Operating Diagrams

Client-Server Architecture: The client communicates with the server via TCP sockets. The server processes requests and accesses a local file (users.txt).

Sequence Diagram (Login): The client sends credentials, the server validates them and returns a success or error response.

Flowchart (Register): The server checks if the username exists. If yes, returns error; otherwise, stores the new user and confirms success.

5. Application Mockups

Main Menu:

1. Register
2. Login
3. Exit

After Login:

1. ECHO
2. GET_INFO
3. Logout

Example Outputs:

✓ Login bem sucedido

✕ Credenciais erradas

Resposta do servidor: Hello

Risk and Testing Manager Report

Risk Management

Risk ID	Description	Impact	Probability	Mitigation Strategy
R1.1	**Data Persistence Failure:** User data not saving after a server crash.	High	Low	Forced the use of fflush() and immediate fclose() after every registration to ensure disk writing.
R1.2	**Blocking Deadlock:** One client hangs, preventing others from connecting (Sequential Model).	Medium	High	Defined a timeout protocol and documented this as a limitation to be solved in Step 3 with select().
R1.3	**Credential Security:** Plain text passwords in users.txt.	Low	High	Accepted for Step 1 as per requirements, but flagged for future hashing implementation.

Testing Plan & Results

Test ID	Requirement	Input Data	Expected Result	Actual Result	Status
TC-101	User Registration	1, user1, pass12	Server saves data and confirms registration.	"✓ Registo feito com sucesso!"	PASSED

TC-102	Valid Login	2, user1, pass12	Access granted to the main menu.	"✓ Login bem sucedido!"	PASSED
TC-103	Invalid Password	2, user1, 0000	Error message and access denied.	"✕ Credenciais erradas!"	PASSED
TC-104	Duplicate User	1, user1, pass12	Prevention of duplicate accounts.	"✕ Utilizador já existe!"	PASSED
TC-105	System Info	2 (GET_INFO)	Server returns version/status.	"Server v1.0 - Running OK"	PASSED
TC-106	Echo Service	1, "Hello World"	Server returns the same string.	"Resposta do servidor: Hello World"	PASSED
TC-107	Session Logout	3, (Logout)	Returns to main menu.	"Logout feito."	PASSED

Introduction

The C-Cord project aims to build a simplified Discord-like communication system in C, progressing from a basic sequential server to a real-time, multiplexed architecture. To contextualize the design choices made throughout this project, this report provides a state-of-the-art analysis of existing communication platforms and the underlying networking models they employ. Understanding these platforms — their features, limitations, and architectural decisions — informs the development path taken in C-Cord.

Existing Communication Platforms

Several platforms have shaped modern real-time communication. The following are the most relevant reference points for C-Cord:

1. Discord

Discord is the primary inspiration for this project. Launched in 2015, it is built on a Client-Server model where a central server manages all messages, user states, and channel memberships. Discord uses WebSockets for persistent, full-duplex connections, allowing the server to push messages to clients instantly — the same behavior C-Cord targets in Phase 3 using select(). Discord is organized into Servers and Channels, which directly maps to the multi-channel support required in feature F8. It also supports role-based access control, analogous to the Admin role in C-Cord. Discord's backend is built in Elixir and Python at scale, but its core communication model, a hub managing multiple client connections simultaneously, is exactly the problem space this project explores at the socket level.

2. IRC (Internet Relay Chat)

IRC, first defined in RFC 1459 (1993), is one of the earliest and most architecturally relevant predecessors. It uses plain TCP connections and a text-based protocol, making it a very close ancestor of what C-Cord implements. IRC introduced the concept of channels (rooms), user authentication via nicknames and passwords, and server-side broadcasting, all concepts directly reflected in the C-Cord feature set. IRC servers historically used select() or poll() to handle multiple clients on a single thread, which is precisely the upgrade that Phase 3 of C-Cord requires. Studying IRC confirms that the architectural choices in this project are well-founded and proven.

3. Slack

Slack, released in 2013, is a workplace communication platform that follows a similar Client-Server model to Discord. It uses a persistent WebSocket connection per client and organizes communication into workspaces and channels. Slack introduced the concept of message persistence, all messages are stored server-side and retrievable later, which maps directly to C-Cord's inbox storage feature (F4: CHECK_INBOX). It also has a well-defined REST API for administrative tasks, paralleling the Admin Management features in F5 and F6. Unlike C-Cord, Slack does not support direct peer-to-peer file transfer, which makes C-Cord's UDP File Transfer feature (F9) a notable point of differentiation.

4. XMPP (Extensible Messaging and Presence Protocol)

XMPP is an open standard protocol (RFC 6120) for real-time messaging over TCP. It is XML-based and supports features such as presence detection (online/offline status), multi-user chat, and direct file transfer via extensions — all reflected in C-Cord's feature set. XMPP's server design uses a stream-based persistent connection model, and many XMPP server implementations rely on non-blocking I/O multiplexing to handle thousands of concurrent sessions, reinforcing the importance of the select()/poll() upgrade in Phase 3 of C-Cord.

Key Technical Concepts

The evolution from a blocking sequential server to a multiplexed real-time server reflects a well-documented progression in network programming. The following concepts are foundational:

Blocking vs. Non-Blocking I/O

In a blocking I/O model, system calls such as accept() and recv() halt execution until data is available. This is the model used in Phases 1 and 2 of C-Cord, and while simple to implement, it fundamentally prevents a single-threaded server from handling multiple clients simultaneously. Real-time platforms like Discord and IRC resolve this at scale through either multi-threading, multi-processing, or, more efficiently, I/O multiplexing.

I/O Multiplexing: select() and poll()

The POSIX functions select() and poll() allow a program to monitor multiple file descriptors simultaneously, only acting when one becomes ready for reading or writing. This is the foundation of Phase 3 in C-Cord. Both functions are widely used in legacy and embedded systems. Modern high-performance servers have evolved to use epoll (Linux) or kqueue (BSD/macOS), which scale more efficiently for thousands of simultaneous connections. For the scope of C-Cord, select() and poll() are appropriate and instructive choices.

TCP vs. UDP

TCP provides reliable, ordered delivery and is used for all messaging features in C-Cord, consistent with how Discord, Slack, and XMPP handle text communication. UDP, used in feature F9 for direct file transfer, trades reliability for lower overhead and is appropriate for latency-sensitive or peer-to-peer transfers where the application layer can handle loss. This duality mirrors real-world platforms: Discord, for instance, uses UDP for voice and video calls while using TCP (via WebSocket) for text.

Development Team Report

Introduction

The C-Cord project aims to develop a simplified communication system inspired by modern platforms such as Discord, focusing on the implementation of a client-server architecture using low-level socket programming in C. In the initial phase of development, the primary objective was to establish the foundational components of the system, namely user registration (F1), while ensuring reliable communication between client and server. This section describes the implementation details of this feature, as well as the structure and design choices adopted during development.

System Architecture and Design

The system follows a classic client-server model, where a central server is responsible for handling all requests and managing system data, while clients act as interfaces for user interaction. Communication between the two is performed using TCP sockets, ensuring reliable and ordered data transmission.
The server operates using a sequential (blocking) model, meaning that it handles one client at a time. While this approach limits scalability, it significantly simplifies the implementation and is appropriate for the initial phase of the project. Each interaction follows a request-response cycle: the client establishes a connection, sends a command, receives a response, and then disconnects.

User Registration (F1)

The user registration feature allows new users to be added to the system through a simple command-based protocol. The client sends a request to the server using the format: REGISTER <username> <password>.
Upon receiving the request, the server reads the incoming data using the recv() system call and processes it by extracting the command and its arguments using string parsing functions such as sscanf(). This enables the server to identify the operation to be performed and the associated user credentials.
To ensure the uniqueness of usernames, the server verifies whether the requested username already exists in a local file (users.txt), which functions as a basic persistent storage mechanism. This verification is carried out by reading the file sequentially and comparing each stored username with the one provided by the client. If a match is found, the server responds with an error message indicating that the user already exists. Otherwise, the new user credentials are appended to the file, and a confirmation message is sent back to the client.
This approach guarantees both data persistence and consistency, preventing duplicate entries while maintaining a simple and efficient storage mechanism.

Server Implementation

The server was implemented using the POSIX socket API and is responsible for managing all communication and data processing tasks. The initialization process involves creating a socket using socket(), binding it to a specific port with bind(), and setting it to listen for incoming connections using listen(). Client connections are accepted using accept(), after which the server receives data through recv() and sends responses using send().
The core logic of the server is structured around an infinite loop that continuously listens for new client connections. For each connection, the server processes a single

request and then closes the connection, maintaining the simplicity of the blocking model.
Command handling is performed through conditional checks based on the parsed input. For example, the server identifies registration requests by comparing the received command string with "REGISTER", allowing it to execute the appropriate logic.

Client Implementation

The client component serves as the interface between the user and the system. It is responsible for collecting user input, formatting commands, and communicating with the server.
The client establishes a connection using socket() and connect(), then sends user-generated commands via send() and waits for a response using recv(). User input is collected using standard input functions such as fgets(), and commands are dynamically constructed using functions like sprintf().
To improve usability, an interactive menu system was implemented, allowing users to navigate between different options such as registration, login, and exit. This menu operates in a loop, enabling continuous interaction without restarting the program.

Data Management

User data is stored in a local file located in the data directory (data/users.txt). Each line in the file represents a user, containing a username and password separated by a space.
The server accesses this file using standard file handling functions (fopen, fscanf, fprintf, fclose) to read existing users and append new ones. This approach provides a simple yet effective form of persistent storage, ensuring that user data is maintained across multiple executions of the server.

User Interface Enhancements

Although not required for the core functionality, several improvements were implemented on the client side to enhance the user experience. These include an interactive menu system, screen clearing using system("clear"), and structured output formatting.
Additionally, terminal colors were introduced using ANSI escape codes to provide visual feedback. Different colors are used to distinguish between successful operations, errors, and general information, improving readability and overall usability.
Basic session management was also implemented on the client side through the use of variables that track whether a user is logged in and store the current username. This allows the interface to adapt dynamically based on the user's state.